AI PROJECT OPERATING SYSTEM

AI-POS v1.0

A Reusable Playbook for Building Decision-Grade AI Systems

Vinay Kumar | KPMG India | April 2026

MODULE 1	Project Initialization System — Scope, charter, stakeholder questions
MODULE 2	AI Engineering Blueprint — Architecture, trust hierarchy, guardrails
MODULE 3	Data Intelligence System — Profiling, validation, analytical patterns
MODULE 4	Evaluation & Learning System — Test harness, failure buckets, iteration
MODULE 5	Product & Domain System — Domain onboarding, roadmap, UX principles

MODULE 1: PROJECT INITIALIZATION SYSTEM

AI-POS

Use this module at the start of every AI project. It prevents scope creep, stakeholder misalignment, and building the wrong thing.

1.1 The Five Scoping Questions (Ask Before Building Anything)

#	Question	Why It Matters	Supply Command AI Example
1	What decision does this AI system support?	Prevents building analytics instead of intelligence	Supplier risk decisions, termination, volume reallocation
2	Who are the users and what are their roles?	Drives RBAC design from day one	3 roles: Ops Manager, CFO, Demand Planner
3	What data exists and how clean is it?	Surfaces data quality issues before architecture is locked	100 shipments, 3 suppliers, 17 misclassified OnTime records
4	What does success look like in 90 days?	Creates measurable exit criteria	32/32 eval test cases passing, human approval on actions
5	What are the constraints (cost, compliance, tools)?	Prevents architecture that cannot be deployed	Single LLM call, SQLite for POC, Streamlit for UI

1.2 Stakeholder Interview Questions

For business stakeholders (non-technical)

- What is the most expensive decision your team gets wrong today?
- How do you currently get the answer to [key business question]?
- What would make you trust an AI recommendation enough to act on it?
- Who should NOT have access to financial data? Why?
- What happens if the AI is wrong? What is the cost of a bad recommendation?

For technical stakeholders

- Where does the data live? What format? How often does it update?
- What ETL or data pipeline exists today?
- Are there regulatory constraints on data residency or model usage?
- What existing tools must we integrate with (ERP, BI, CRM)?
- What is the latency budget for a query response?

1.3 Success Metrics Framework

Layer	Metric	Measurement Method	Threshold
Accuracy	Query answer correct rate	Automated eval harness vs golden answers	>= 90%
Trust	Confidence score calibration	High conf answers more accurate than low	Monotonic relationship
Speed	P99 pipeline latency	Eval harness timing	< 10s for decision queries
Cost	Cost per query	LLM token tracking	< $0.001 per query
Safety	RBAC bypass rate	Audit log review	0 bypasses
Adoption	User queries per day	Decision log count	> 50 queries/day at launch

1.4 PROJECT_CHARTER Template

TEMPLATE — COPY AND FILL FOR EVERY NEW AI PROJECT

Project Name: _____Domain: _____ | Client: _____ | Date: _____PROBLEM STATEMENT: [One sentence — what decision is broken today?]AI SYSTEM PURPOSE: [One sentence — what will the AI do differently?]USERS & ROLES: [Role 1] — [access level] | [Role 2] — [access level]DATA SOURCES: [Table/file name] | [rows] | [update frequency] | [quality flag]SUCCESS CRITERIA: [Metric] >= [threshold] by [date]OUT OF SCOPE: [What this system will NOT do — equally important]CONSTRAINTS: [Cost / compliance / tools / timeline]SIGN-OFF: [Stakeholder name + role]

1.5 What Seniors Will Ask — Module 1

- How did you decide what to build? → Show the 5 scoping questions answered for this project
- Who are the users? → Name the roles, their access levels, and what they cannot see
- How do you know it is working? → Show the success metrics and current scores
- What is out of scope? → Name 3 things you deliberately did not build and why

2.1 The Trust Hierarchy (Non-Negotiable)

CORE PRINCIPLE

DB > RAG > LLM. Structured data from a verified SQL template is always more trustworthy than retrieved document context, which is always more trustworthy than LLM inference. Never allow a lower-trust source to override a higher-trust source.

Source	Trust Level	Confidence	When to Use	Supply Command AI Example
Hardcoded SQL Template	Highest	0.90	Known metric queries — counts, rates, totals	delay rate, total cost, OTD
LLM-generated SQL	High	0.70 max	Novel queries not in template library	New dimension never seen before
RAG — verified chunk	Medium	0.80–0.86	External context, benchmarks, PDFs	Industry OTD 87% from Annual Report
LLM inference only	Low	0.60 max	Formatting and synthesis only	Executive agent final output
LLM with no context	Lowest	Never use	Never for data retrieval	Blocked by architecture

2.2 Standard Agent Architecture

Every AI system should implement this pipeline regardless of domain:

Layer	Component	Responsibility	Key Rule
Gate 1	Input Guardrails	RBAC check, injection scan, financial keyword block	Runs BEFORE planning. Fail-fast.
Planning	Planning Agent	Intent, metric, dimension, time filter extraction	Deterministic routing. No LLM for routing decisions.
Execution	DB Agent	SQL template matching and execution	80% hardcoded templates. 20% LLM fallback max.
Execution	RAG Agent	Vector search on knowledge base	Only for EXPLANATION and DECISION queries. Never METRIC.
Execution	ROI Agent	Pure Python financial calculations	0 LLM calls. Math only.

Layer	Component	Responsibility	Key Rule
Gate 2	Output Guardrails	Hallucination check, cross-metric filter, confidence	Runs AFTER agents. Strips contamination.
Approval	Human-in-Loop	Pause on material actions	Fires on action keywords only. Not dollar amounts.
Output	Executive Agent	LLM formatting of DB findings	1 LLM call total per query. Formatting only.

2.3 Query Classification System

Every query must be classified BEFORE routing. This is the most important design decision in any AI system.

Query Type	Intent Examples	Agents Called	LLM Calls	Max Response Length
METRIC_QUERY	Count, rate, total, highest, lowest	DB only	0 (bypass)	1–2 sentences
EXPLANATION_QUERY	Why, how, compare, benchmark	DB + RAG	1 (format)	3–4 sentences
DECISION_QUERY	Should we, recommend, what to do	DB + RAG + ROI	1 (format)	Structured: Issue / Rec / Risk
WHATIF_QUERY	What if, simulate, if rate drops	ROI only	0 (Python math)	Structured simulation output

2.4 Semantic Layer — The Hallucination Prevention Foundation

Before any LLM call, inject this context. This is what prevents column name hallucination.

- Canonical metric definitions — "delayed shipment" = status = Delayed. Never delay_days > 0.
- Exact column names from confirmed schema — expedited_ship_usd not expedited_cost
- Allowed dimension values — region: North, South, West. Not "Northeast" or "Central"
- Business rules — OTD and delay rate are different metrics. Never substitute.
- Time grain — year filter applies to financial_impact.year column, not shipment_date substring

2.5 Common Failure Patterns (From Supply Command AI)

Failure	Root Cause	Fix	Detection
Wrong metric returned (OTD vs delay rate)	Planner had no metric disambiguation	Add detected_metric field to plan dict	Eval test: expected contains "15%" not "80%"
Wrong dimension (supplier vs region)	No detected_dimension in planner	Strict "region" keyword → GROUP BY region	Eval test: expected contains "South" not "SUP003"
Year-specific query returns cumulative	No time filter extraction	extract_time_filter() before intent detection	Eval test: 2022 cost = $5.7M not $17.4M
No data available (data exists)	Missing SQL template	Add template + keyword routing	Eval test: confidence = 0% is always a bug
Numbers rendered vertically	Python f-string formatting with newlines	clean_finding_text() at DB agent output layer	Eval test: must_not_contain newline patterns
Investigation plan on simple query	Metric queries routed to full pipeline	SIMPLE_METRIC bypass in executive agent	Eval test: agents_used = 1 for metric queries
RBAC bypass via re-routing	Access check ran after planning	Move RBAC check to first node in pipeline	Eval test: Demand Planner + financial = blocked

2.6 ARCHITECTURE Template

TEMPLATE — FILL FOR EVERY NEW AI PROJECT

QUERY TYPES: [List your query types and what agents fire for each]TRUST HIERARCHY: DB templates > [your retrieval method] > LLM inferenceRBAC ROLES: [Role] → [blocked tables/intents]HUMAN APPROVAL TRIGGERS: [Action keywords that pause pipeline]CONFIDENCE CALIBRATION: Template = [X] | LLM SQL = [Y] | RAG = [Z]SEMANTIC LAYER CONTENTS: [Canonical definitions for your domain]EVALUATION THRESHOLD: Pass if [N/M] test cases passPRODUCTION PATH: [Current tech] → [Production tech] — one change needed

2.7 What Seniors Will Ask — Module 2

- How do you prevent hallucination? → Three layers: SQL templates, semantic layer, output guardrails
- Why multiple agents vs one LLM? → Cost and determinism. DB agent: 0 LLM calls. Total: 1 call per query.
- Is the human-in-loop real? → LangGraph actually pauses. LLM not called until approval received.
- How is confidence calibrated? → From SQL source type and row count, not LLM self-assessment

- Can it scale? → Every component has a named production replacement requiring zero re-architecture

3.1 Data Profiling Checklist (Run Before Any Architecture Decision)

☐	Row count per table — is volume realistic for the business (100 shipments over 2 years is synthetic)
☐	Column data types — confirm TEXT vs INTEGER vs REAL match business expectations
☐	Null rates per column — >5% warn, >20% alert, >50% unusable for that column
☐	Distinct value counts — categorical columns should have bounded cardinality
☐	Date range coverage — does the data span the period needed for analysis?
☐	Referential integrity — FK relationships clean (no orphaned supplier IDs in shipments)
☐	Business rule cross-checks — status=OnTime but delay_days>0 is a domain-specific violation
☐	Aggregation sanity — does SUM(annual costs) equal reported total?
☐	Metric consistency — is "delayed" defined the same way across all tables?
☐	Time grain alignment — shipments at row level, financials at monthly level — understand the join implication

3.2 Data Validation Rules

Cross-column rules (most commonly missed)

Rule	SQL Check	Supply Command AI Finding	Severity
Status vs delay_days	status='OnTime' AND delay_days>0	17 shipments violated — true delayed may be 32 not 15	HIGH
Delivery date vs status	actual_delivery_date > expected AND status=OnTime	17 shipments delivered late but marked OnTime	HIGH
Missing delivery on completed	actual_delivery_date IS NULL AND status NOT IN ('In_Transit','Cancelled')	0 violations — clean	CHECK
Negative financial values	total_sc_cost_usd < 0	0 violations — clean	CHECK
SLA target out of range	sla_on_time_target_pct < 0 OR > 100	0 violations — clean	CHECK

3.3 Analytical Patterns — What Questions to Always Answer

For any supply chain or operations dataset, always build templates for:

Pattern	Question Type	SQL Pattern	Output Format
Fleet scalar	What is the overall rate?	SELECT COUNT(*) / SUM(case)	Single number + context
Dimension ranking	Which [dim] has highest [metric]?	GROUP BY dim ORDER BY metric DESC LIMIT 1	Entity + value + rank
Comparison	Compare [metric] across all [dim]	GROUP BY dim ORDER BY metric DESC	Ranked list
Time filter	What is [metric] for [year]?	WHERE year = {year}	Single value + year label
Year-on-year trend	How did [metric] change 2022-2024?	GROUP BY year ORDER BY year	Year | value | change%
Value aggregation	What is total/average [value]?	SUM(value_col) / AVG(value_col)	Dollar amount formatted
Date span	What time period does data cover?	MIN(date) / MAX(date)	From [date] to [date]
Anomaly	What is inconsistent?	Cross-column business rule SQL	Count + description + severity

3.4 DATA_PLAYBOOK Template

TEMPLATE — COMPLETE BEFORE FIRST SPRINT

CANONICAL DEFINITIONS: [Metric A] = [exact SQL condition] [Metric B] = [exact SQL condition] NOTE: Never substitute [A] for [B]CONFIRMED COLUMN NAMES (from PRAGMA table_info): [table].[column] → [business meaning]DATA QUALITY ISSUES FOUND: [Issue] | [Count] | [Severity] | [Impact on AI answers]ANALYTICAL TEMPLATES NEEDED: [Query pattern] → [SQL template name]HEALTH SCORE: [X]% ([N] checks passing of [M] total)DATA SIGN-OFF: These definitions are locked. Any change requires re-running eval harness.

3.5 What Seniors Will Ask — Module 3

- How do you know the data is reliable? → Data health agent runs pre-flight checks on every load. Current score: 70% with 2 HIGH issues surfaced.
- What are the data quality issues? → 17 shipments misclassified. System flags this and appends caveats to relevant answers.
- Is this production-ready data? → No — synthetic dataset. Architecture is production-ready. Production requires ERP connectors.

- How do you handle new data? → Canonical definitions are locked. New data runs through same validation rules. Health score recalculates on load.

MODULE 4: EVALUATION & LEARNING SYSTEM

AI-POS

THE MOST IMPORTANT MODULE

Manual testing is not scalable. The moment you have more than 10 queries to validate, you need an automated eval harness. Build this before you build anything else. Every fix should be validated by running the harness, not by asking questions manually.

4.1 Test Case Design Rules

- Every test case must have: query, role, must_contain list, must_not_contain list, category
- must_contain checks the correct answer is present — not just any answer
- must_not_contain catches wrong metric substitution, broken formatting, hallucinated text
- Categories must match your failure buckets — fix by category, not by question
- Locked expected values must be verified against DB directly before being locked
- Add a test case for every bug you fix — regression prevention

4.2 The Five Failure Buckets

Every AI system failure maps to exactly one of these. Never say "the AI is weak." Say which bucket.

Bucket	What It Covers	Fix Location	Supply Command AI Examples
Planner	Wrong intent, metric, dimension, time filter	planning_agent.py	OTD returned for delay rate, region→supplier confusion, year filter ignored
DB Templates	Missing templates, wrong SQL, wrong column names	db_agent.py	Total shipment value = no data, AI savings = no data, annual vs cumulative
Executive Output	Verbosity, formatting, cross-metric contamination	executive_agent.py	Numbers rendered vertically, recommendations added to metric queries
RBAC / Decision	Access bypass, wrong approval trigger	guardrails.py	Demand Planner seeing financial data, approval on dollar amount not action
Data Quality	Wrong answers from dirty data	data_health_agent.py	17 misclassified shipments affecting delay counts

4.3 The Evaluation Loop

1. Lock expected answers for all test cases in test_cases.py
2. Run eval harness: python3 evaluation/run_eval.py

3. Read category summary — fix only categories below threshold
4. Apply targeted fix to one category at a time
5. Re-run harness — verify category improved without regression in others
6. Push to production only when overall pass rate meets threshold
7. Add new test cases for any failure found in production

4.4 Eval Harness Structure

Every AI project must have this file structure:

- evaluation/test_cases.py — locked expected answers, never change without re-running
- evaluation/run_eval.py — automated runner, produces category report
- evaluation/eval_results_[timestamp].json — results archive for trend tracking
- evaluation/golden_test_set.py — extended Q&A pairs for domain coverage

4.5 Pass/Fail Criteria

Status	Condition	Action
Demo Ready	>= 90% overall, 100% on basic_metrics and rbac	Deploy. Rehearse demo whitelist.
Near Ready	75-89% overall, no critical failures	Fix failing categories only. Re-run.
Not Ready	< 75% overall OR any rbac failure	Stop. Fix root causes by bucket. Re-run.
Regression	Previously passing test now failing	Revert last change. Fix properly.

4.6 EVAL_SYSTEM Template

TEMPLATE — MINIMUM TEST CASE SET FOR ANY AI PROJECT

CATEGORY: basic_metrics (query, role, ["expected_value"], [], "basic_metrics") Cover: count, rate, total, average, date spanCATEGORY: dimension_analysis Cover: by supplier, by region, by category, ranked listCATEGORY: time_filter Cover: year-specific, month-specific, year-on-yearCATEGORY: rbac Cover: each restricted role + each blocked query type must_contain: ["restricted", "Access"]CATEGORY: formatting must_not_contain: ["\n,\n", "expeditedfreight", "MergedWords"]CATEGORY: decision Cover: human approval fires on action keywordsTHRESHOLD: >= 90% overall required for demo/production

4.7 What Seniors Will Ask — Module 4

- How do you know it works? → Automated eval harness. 32/32 passing as of this morning.
- What if it breaks after a change? → Harness runs before every deployment. Regression = revert.
- How do you handle new query types? → Add test case in test_cases.py. Run harness. Fix if red.

- What is your test coverage? → [N] cases across [M] categories covering all query types and all roles

5.1 Domain Onboarding Framework (Any Industry)

Use this checklist when entering a new domain (pharma, energy, consumer, logistics):

☐	Identify the 5 most expensive decisions made in this domain (these are your use cases)
☐	Map who makes each decision, with what data, on what frequency
☐	Find the single most important metric — the one number everyone cares about
☐	Identify the canonical definitions — what does "delayed", "at-risk", "critical" mean here
☐	Find the benchmark — industry standard that makes your metric meaningful
☐	Identify the roles and what each role must not see (RBAC design)
☐	Find the action keywords — what words trigger a real-world action (terminate, expedite, escalate)
☐	Map the data sources — what tables, what columns, what grain, what update frequency
☐	Identify the domain-specific data quality rules — what cross-column rules should never be violated
☐	Find a domain expert who can validate AI answers — do not build without a domain validator

5.2 AI Product Roadmap Structure

Phase	Focus	Deliverable	Exit Criteria
Phase 1 — Foundation	Data, architecture, basic metrics	Working pipeline on verified data	5 core queries passing at 90%+ confidence
Phase 2 — Intelligence	RAG, decision queries, guardrails	Explainable recommendations with sources	RBAC, human-in-loop, hallucination prevention live
Phase 3 — Proactive	Alerts, simulation, what-if	System initiates insight before user asks	Proactive alerts + what-if simulation live
Phase 4 — Learning	Feedback loop, eval harness, fine-tuning	System improves with use	Automated eval, feedback score tracked
Phase 5 — Scale	Production infra, ERP connectors, compliance	Enterprise deployment	PostgreSQL, Pinecone, SOC 2 path defined

5.3 Prioritization Framework

When deciding what to build next, score each feature on three dimensions:

Dimension	Score 1	Score 2	Score 3	Weight
Demo Impact	Not visible in demo	Visible but not wow	Changes demo narrative completely	40%
Fix vs Feature	New feature, no bugs fixed	Fixes one bug + adds value	Fixes critical bug	40%
Effort	> 2 days	4-8 hours	< 2 hours	20%

Build only items scoring > 6/10 weighted. Everything else is Phase 2.

5.4 UX Principles for AI Systems

Principle 1: Show your work

Every AI answer must show: source, confidence, SQL used. Users trust systems they can audit.

Supply Command AI: "View Sources, SQL & Evidence" expander on every answer.

Principle 2: Fail loudly, not silently

A system that says "no data available" is better than one that returns a plausible wrong answer.

Supply Command AI: Confidence 0% + caveat is better than hallucinated $17M when $5.7M expected.

Principle 3: Role-appropriate interface

The same data looks different to a CFO vs a Demand Planner. The UI must enforce this, not just the backend.

Supply Command AI: Financial KPI cards show "Restricted" for Demand Planner — not just an error after query.

Principle 4: Proactive over reactive

The highest-value AI insight is the one the user did not know to ask for.

Supply Command AI: Proactive alerts fire on dashboard load. No query needed.

Principle 5: Short answers for metrics, structured answers for decisions

Metric query = 1-2 sentences maximum. Decision query = Issue / Recommendation / Risk structure.

Supply Command AI: METRIC_QUERY enforces 2-sentence limit. DECISION_QUERY uses structured bullets.

5.5 The "Modern AI System" Checklist

A system earns the label "Modern AI" when it has ALL of these:

☐	Deterministic routing — query type known before any agent fires
☐	Trust hierarchy enforced — DB > RAG > LLM, never reversed
☐	RBAC at pre-planning gate — security before business logic
☐	Human-in-loop on material actions — LLM cannot execute actions autonomously
☐	Proactive alerting — system surfaces insight without being asked
☐	Confidence calibration — from source type, not LLM self-report
☐	Automated evaluation harness — pass rate tracked before every deployment
☐	Data health monitoring — data quality issues surfaced and flagged in answers
☐	Output contamination filter — metric answers contain only the asked metric
☐	Explainability — every answer shows source, SQL, and confidence
☐	Cost transparency — cost per query tracked and optimised
☐	Production path defined — every POC component has a named production replacement

5.6 PRODUCT_OS Template

TEMPLATE — REVIEW MONTHLY FOR ANY AI PRODUCT

DOMAIN METRIC (the one number that matters): _____DOMAIN BENCHMARK (industry standard): _____CANONICAL DEFINITIONS: [Locked in data playbook]CURRENT PHASE: [1 / 2 / 3 / 4 / 5]EVAL PASS RATE: [X/Y] ([Z%])COST PER QUERY: $_____DEMO WHITELIST: [Query categories with 100% pass rate]DO NOT DEMO: [Query categories below 70%]NEXT SPRINT PRIORITY (score > 6/10 only): 1. [Feature] — Impact [X] | Fix [Y] | Effort [Z] = Score [total]FALLBACK LINE FOR DEMO: "That path is in our next sprint. Let me show you [whitelist query]."VERSION 2 ROADMAP: [Conversational memory | Multi-LLM routing | ERP connectors | MCP integration]

5.7 What Seniors Will Ask — Module 5

- How does this differ from a BI tool? → BI shows what happened. This explains why, recommends what to do, and controls who can act.
- What is the business case? → [Domain metric] gap of [X] points = $[Y] annual exposure. System surfaces this proactively.
- What is the roadmap? → 5 phases. Currently in Phase [N]. Next: [specific feature].
- Can you show me something it does that ChatGPT cannot? → RBAC, human approval, proactive alerts, DB-grounded answers with SQL shown.
- What domain expertise do you bring? → [X] years in [domain]. I can validate AI outputs in ways a pure AI practitioner cannot.

QUICK REFERENCE — Under-Pressure Cheat Sheet

AI-POS

When a Demo Goes Wrong

Situation	Response
System gives wrong answer	"That query is in our validation pipeline. Let me show you a verified flow instead."
Confidence shows 60%	"60% means LLM-generated SQL was used — the system is being transparent about its certainty."
System is slow	"Decision queries run all three agents. Metric queries run in under 20ms — let me show you."
Senior asks about limitation	"You are right — [limitation] is on our roadmap. Today I want to show [strength]."
Data question	"The dataset is synthetic and I acknowledge that. The architecture is what is being validated."

The Five Questions Every Senior Asks

Question	One-line Answer
How do you prevent hallucination?	Three layers: SQL templates, semantic layer, output guardrails. LLM never retrieves data.
Is the human approval real?	LangGraph actually pauses. LLM not called until human approves. Not cosmetic.
Can it scale?	Every component has a named production replacement requiring zero re-architecture.
What is the ROI?	[Domain-specific: cite your financial model or simulation output]
How is it different from ChatGPT?	RBAC, human approval, DB-grounded answers, proactive alerts, confidence calibration.

The AI-POS Formula

THE FORMULA — REPEAT UNTIL MEMORISED

Semantic layer → Deterministic retrieval → Output control → Evaluation loopSemantic layer: Lock canonical definitions before building anythingDeterministic: SQL templates first. RAG second. LLM last.Output control: 1-2 sentences for metrics. Structure for decisions. Never mix.Evaluation loop: Automated harness. Fix by bucket. Deploy only when green.Domain expertise + AI architecture + evaluation discipline = Decision-grade AI